

TPU_CTRL 模块详设

1. 概述

TPU_CTRL 模块是 TPU_TOP 内部的核心调度单元，负责协调整个 TPU 的数据流动，管理计算任务的生命周期，并实现跨时钟域的信号同步。

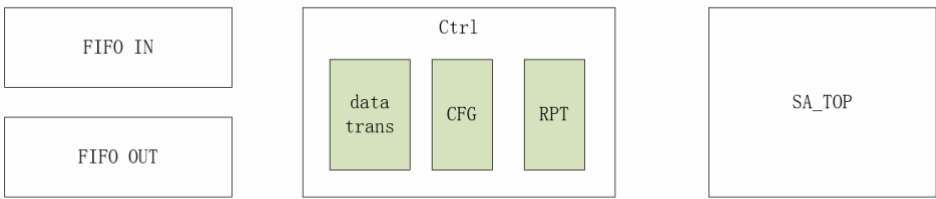
2. 功能特性

TPU_CTRL 的主要功能包括：

- 同步配置信号：将 AHB 时钟域(100MHz)的静态配置与动态触发信号同步到系统时钟域(400MHz)。
- 任务调度：作为控制中心，生成 rd_start 和 wr_start 脉冲，调度 AXI_TOP 执行片外数据的读取与写回。
- 数据吞吐：支持从 FIFO_IN 读取 128-bit 宽字数据，并根据计算精度配置 (INT4/INT8/FP16/FP32) 对不同行数据施加特定延迟偏移，确保数据呈阶梯状对齐，满足脉动阵列的计算需求。
- 状态上报：实时监控计算进度与状态异常，将计算完成及内部状态信号从系统时钟域同步回 AHB 时钟域，供 CPU 通过寄存器查询。

3. 总体架构

TPU_CTRL 模块内部划分为三个子模块，分别负责配置同步、数据处理与状态上报，共同协调 AXI 搬运与脉动计算的流水线逻辑：



- CFG(配置同步模块)：
负责锁存和同步来自 CPU 的静态配置信号(如使能、精度模式)与动态触发信号(如启动、复位)。它将信号从 AHB 时钟域安全引入系统时钟域。
- data_trans(数据转换模块)：
连接 FIFO 与 SA_TOP，将 128-bit FIFO 输出拆解为 4 路 32-bit 数据流。该模块通过多级移位寄存器链，为各行数据提供可配置的延迟，确保进入脉动阵列的数据在时域上满足阶梯对齐要求。
- RPT(状态上报模块)：
负责计算任务的状态反馈。它实时监控顶层 FSM 状态与内部计数器，将计算完成、进度及异常信息从系统时钟域同步回 AHB 时钟域。这些状态最终通过 register map 呈现给 CPU。

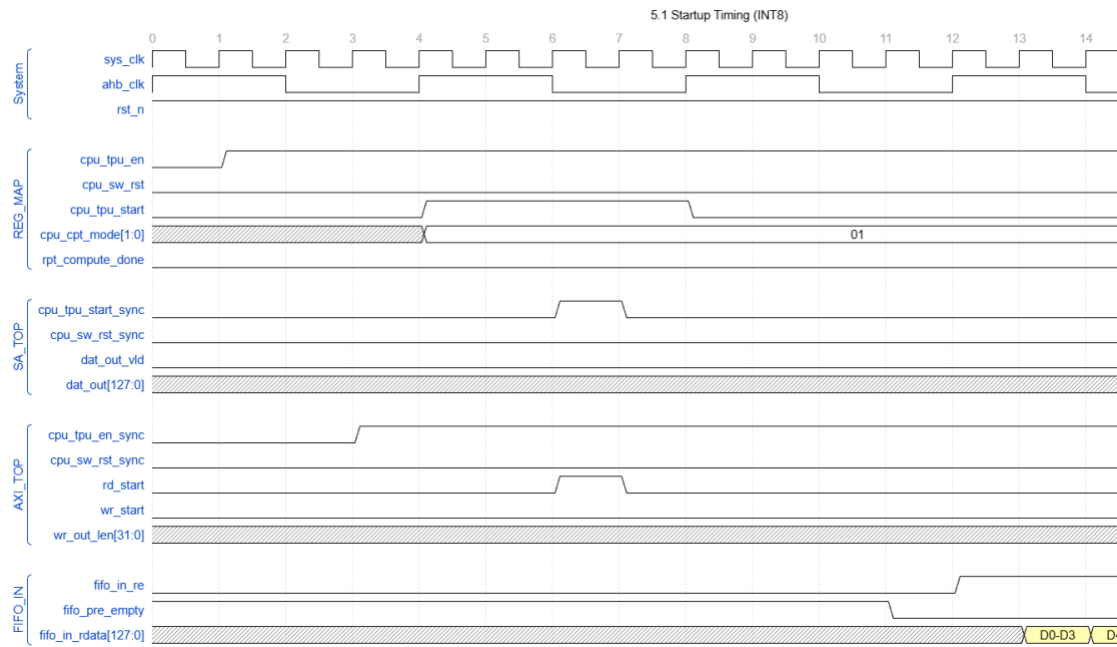
4. 接口定义

System Interface			
sys_clk	1	input	系统时钟，时钟频率 400MHz
ahb_clk	1	input	AHB 总线时钟，时钟频率 100MHz
rst_n	1	input	异步复位，低电平有效
REG Map Interface			
cpu_tpu_en	1	input	TPU 模块使能信号，静态配置
cpu_sw_rst	1	input	TPU 模块软复位信号，动态配置
cpu_tpu_start	1	input	TPU 模块启动信号，动态配置
cpu_cpt_mode	2	input	TPU 模块当前轮计算精度配置信号，静态配置 00: INT4 01: INT8 10: FP16 11: FP32
rpt_compute_done	1	output	TPU 模块计算完成指示信号，上报寄存器
rpt_loop_count	32	output	实时显示当前循环计数，上报寄存器
rpt_fsm_state	3	output	TPU 内部状态机当前状态，上报寄存器
rpt_error_state	1	output	TPU 计算 error，上报寄存器
SA Top Interface			
cpu_sw_rst_sync	1	output	同步到 sys_clk 域的软复位信号
cpu_tpu_start_sync	1	output	同步到 sys_clk 域的 start 信号
dat_out_vld	1	output	输出数据有效信号
dat_out	COL*DW_OUT	output	送入脉动阵列的输入数据，128 bit
AXI Top Interface			
cpu_tpu_en_sync	1	output	同步后 TPU 使能信号
cpu_sw_rst_sync	1	output	同步后的软复位信号
rd_start	1	output	读任务启动脉冲
wr_start	1	output	写任务启动脉冲
wr_out_len	32	output	写数据拍数
FIFO_IN Interface			
fifo_in_re	1	output	输入 FIFO 读使能
fifo_pre_empty	1	input	输入 FIFO 预空信号
fifo_in_rdata	128	input	输入 FIFO 读数据

5. 传输时序说明

5.1 启动时序

CPU 配置 `cpu_tpu_en` 和 `cpu_cpt_mode` 后，发出 `cpu_tpu_start` 脉冲。经 2 拍 `sys_clk` 同步后产生 `cpu_tpu_start_sync`，同拍触发 `rd_start` 通知 AXI_TOP 开始读数据。AXI_TOP 将数据写入 FIFO_IN 后 `fifo_pre_empty` 拉低，FSM 进入 CALC 状态开始读取。

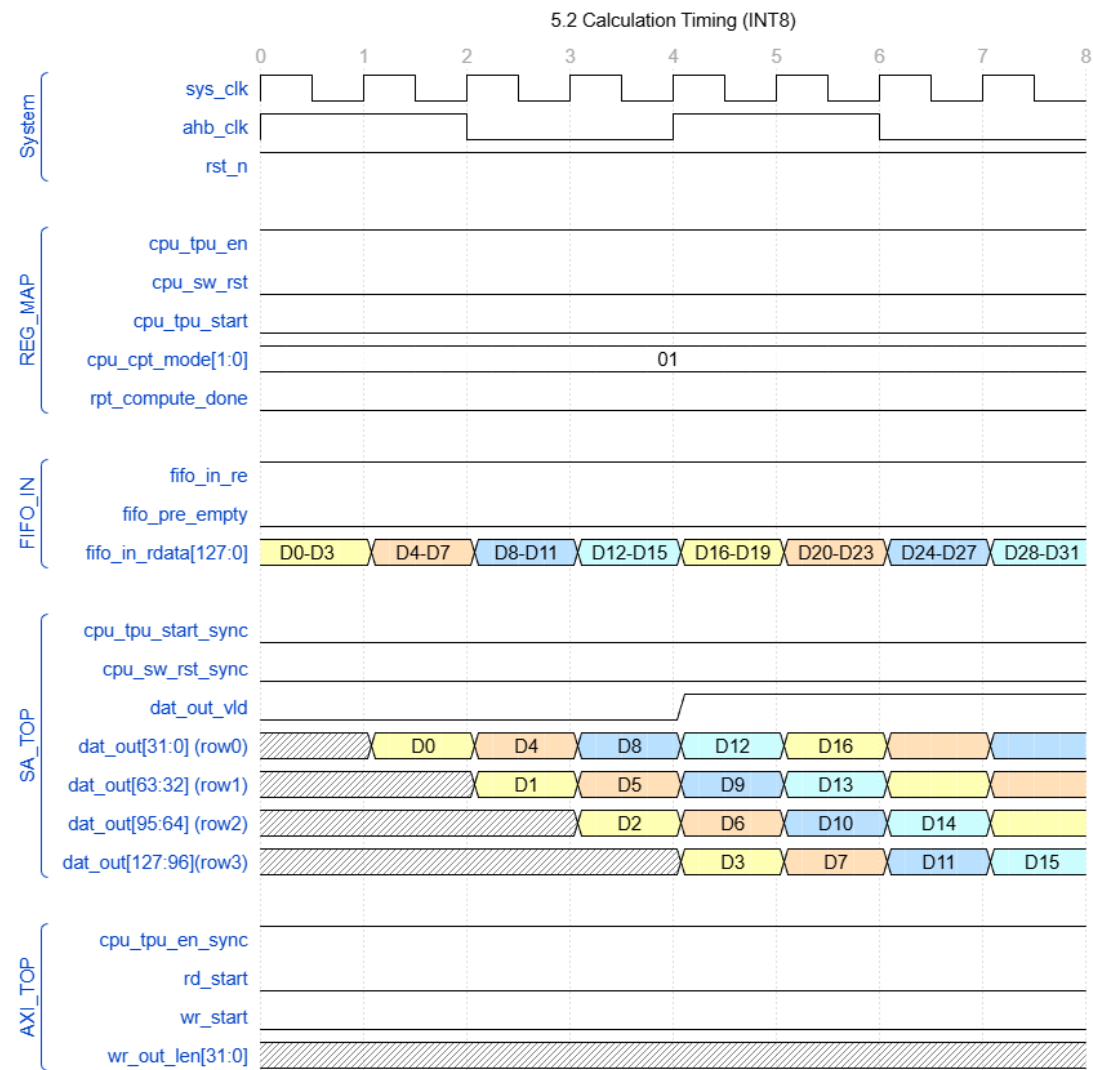


关键时序：

- `cpu_tpu_en` → `cpu_tpu_en_sync`: 2 拍 `sys_clk`
- `cpu_tpu_start` → `cpu_tpu_start_sync`: 2 拍 `sys_clk`
- `cpu_tpu_start_sync` 与 `rd_start` 同拍有效
- `fifo_pre_empty` 拉低后，下一拍 `fifo_in_re` 拉高
- `fifo_in_re` 拉高后，下一拍 `fifo_in_rdata` 有效

5.2 计算时序

FIFO 每拍输出 128bit 数据，data_trans 对各行施加 Skew 延迟后输出到 SA_TOP。

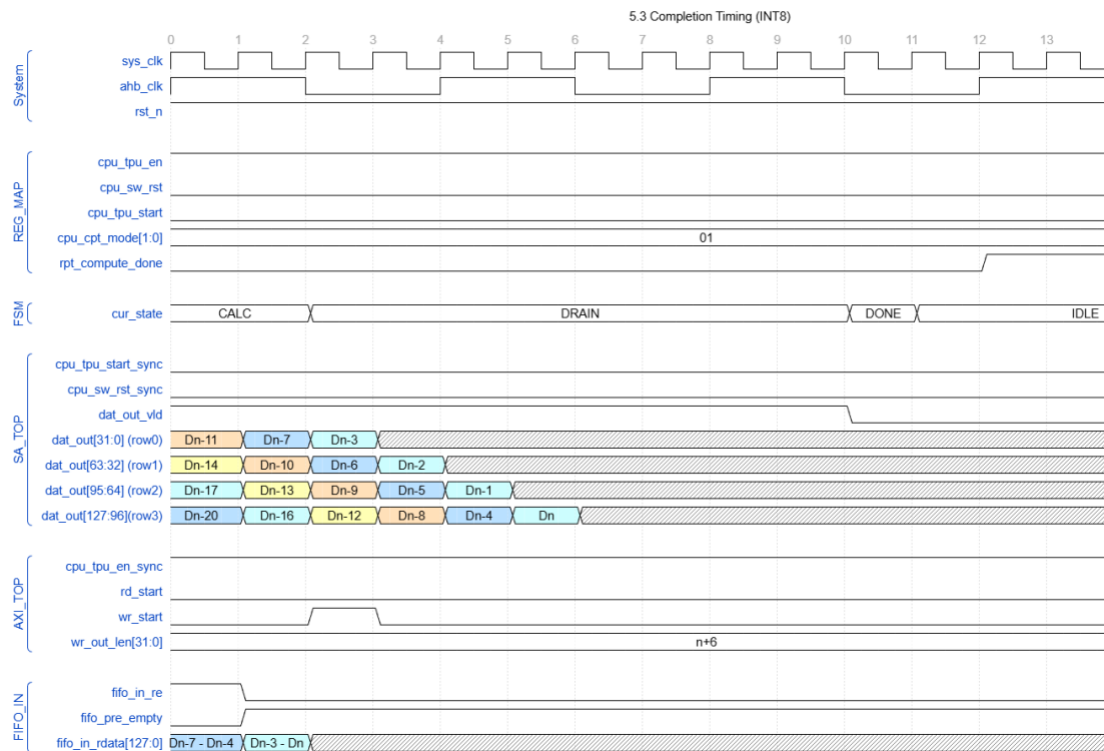


INT8 时序:

- T1: row0=D0 (delay=1)
- T2: row1=D1 (delay=2)
- T3: row2=D2 (delay=3)
- T4: row3=D3 (delay=4), dat_out_vld 拉高
- T5: row0=D4, row1=D5... 继续流水

5.3 完成时序

FIFO 读空后 `fifo_pre_empty` 拉高，FSM 跳转到 DRAIN 状态，同时发出 `wr_start` 脉冲。等待 `drain_max` 个周期排空 SA 阵列残余数据后完成。



关键时序：

- T1: 读入最后一组 `fifo_r_data`, `fifo_pre_empty` 拉高, `fifo_in_re` 拉低
- T2: FSM 进入 DRAIN, `wr_start` 拉高, `row0=Dn-3`
- T3-T5: 最后一组数据依次通过: T3: `row1=Dn-2`, T4: `row2=Dn-1`, T5: `row3=Dn`
- T6-T9: 系统维持 `dat_out_vld` 为高, 持续 `drain_max` 个周期以排空脉动阵列流水线。在 T9 结束时, `drain_cnt` 达标, `dat_out_vld` 拉低。
- T10: FSM 进 DONE
- T11: FSM 进入 IDLE

6. 详细逻辑实现

6.0 状态机

TPU_CTRL 模块采用计数器驱动的有限状态机，通过四个状态管理计算任务的生命周期：

- IDLE：初始状态，持续监控同步后的启动信号。一旦使能并检测到启动脉冲，立即产生 rd_start 触发 AXI 搬运数据，并跳转至 CALC 状态。
- CALC：计算状态，模块通过判断 FIFO 非空状态产生 fifo_in_re 从 FIFO_IN 取数。同时 input_cnt 实时累加取数拍数，作为后续写回长度 wr_out_len 的基准。当监测到 FIFO 变空且输入计数大于零时，判定输入结束，产生 wr_start 并跳转至 DRAIN 阶段。
- DRAIN：数据流收尾状态，此时 fifo_in_re 停止拉数，系统进入排空计时。通过维持 data_out_vld 有效，确保已经进入流水线的参与数据完成运算并流出脉动阵列。排空周期由 $\text{drain_max} = (\text{ROW} - 1) * L + (\text{COL} - 1)$ 决定。
- DONE：任务结束状态，当 drain_cnt 达到 drain_max 后，产生 compute_done 脉冲。该信号通过 CTRL_RPT 模块同步回 AHB 时钟域以更新寄存器状态，任务完成后 FSM 返回 IDLE 状态准备下一轮调度。

6.1 ctrl_cfg 模块

Ctrl_cfg 模块负责处理跨时钟域信号，确保 100MHz AHB 域的控制信号稳定进入 400MHz 系统时钟域：

- 静态配置信号：针对静态配置信号 cpu_tpu_en 和 cpu_cpt_mode，采用两级同步器打拍处理，消除亚稳态。
- 动态脉冲信号：针对动态触发信号 cpu_tpu_start 和 cpu_sw_rst，模块在采样后执行边沿检测逻辑 ($d1 \ \& \ \sim d2$)，提取出单周期的同步脉冲，作为系统启动与复位的唯一有效触发信号。同步后的软复位信号 cpu_sw_rst_sync 分发至 AXI_TOP 和 SA_TOP。

6.2 ctrl_data_trans 模块

Ctrl_data_trans 模块作为数据路径的核心，负责将 128bit 输入流转换为脉动阵列需要的并行错位流：

- 数据拆分：FIFO_IN 输入数据位宽为 128bit，将单拍 128-bit 数据拆分为 4 路 32-bit 数据流 (Row 0-3)。当 fifo_in_re 有效时，数据进入移位寄存器组进行时域偏移处理。
- 多精度延迟：为了让数据呈阶梯状准时进入脉动阵列，模块为不同行设计了基于移位寄存器阵列的可调延迟。偏移量由当前计算精度决定，具体为 INT4/8: L=1; FP16: L=3; FP32: L=4。根据偏移公式，Row i 的输出选择移位寄存器中的第 $\text{Delay}_i = i * L - 1$ 个 Tap。
- 数据同步：输出信号 data_out_vld 与延迟最久的 row3 数据流对齐。

6.3 ctrl_rpt 模块

Ctrl_rpt 模块负责将系统时钟域 (400MHz) 的实时计算状态同步至 AHB 时钟域 (100MHz)，供 CPU 通过 Register Map 进行监测：

- 任务完成上报 (rpt_done)：将 DONE 脉冲同步至 AHB 时钟域，将 rpt_compute_done 寄存器置 1。
- 计算循环上报 (rpt_cnt)：将内部计数器同步到 rpt_loop_count 寄存器，实时反馈数据处理进度。

- 状态机上报(rpt_fsm): 将顶层 FSM 的当前状态(IDLE/CALC/DRAIN/DONE)同步处理后输出到 rpt_fsm_state 寄存器。
- 异常上报(rpt_cpt_error): 监测计算过程中的异常情况, 同步至 rpt_cpt_error 寄存器。